WIKIPEDIA 25
25 years of the free encyclopedia

# Log4j

Apache **Log4j** is a Java-based logging utility originally written by Ceki Gülcü. It is part of the Apache Logging Services, a project of the Apache Software Foundation. Log4j is one of several Java logging frameworks.

Gülcü has since created SLF4J, Reload4j,[4] and Logback[5] which are alternatives to Log4j.[6]

The Apache Log4j team developed Log4j 2[7] in response to the problems of Log4j 1.2, 1.3, `java.util.logging` and Logback, addressing issues which appeared in those frameworks.[8] In addition, Log4j 2 offered a plugin architecture which makes it more extensible than its predecessor. Log4j 2 is not backwards compatible with 1.x versions,[9] although an "adapter" is available. On August 5, 2015, the Apache Logging Services Project Management Committee announced that Log4j 1 had reached end of life and that users of Log4j 1 were advised to upgrade to Apache Log4j 2.[10] On January 12, 2022, a forked and renamed log4j version 1.2 was released by Ceki Gülcü as Reload4j version 1.2.18.0 with the aim of fixing the most urgent issues in log4j 1.2.17 that had accumulated since its release in 2013.[11]

On December 9, 2021, a zero-day vulnerability involving arbitrary code execution in Log4j 2 was published by the Alibaba Cloud Security Team and given the descriptor "Log4Shell".[12] It has been characterized by Tenable as "the single biggest, most critical vulnerability of the last decade".[13]

**Apache Log4j**

| | |
|---|---|
| **Developer** | Apache Software Foundation |
| **Initial release** | January 8, 2001[1] |
| **Stable release** | 2.26.0[2] / 7 May 2026[3] |
| **Written in** | Java |
| **Operating system** | Cross-platform |
| **Type** | Logging |
| **License** | Apache License 2.0 |
| **Website** | logging.apache.org/log4j/2.x/ (https://logging.apache.org/log4j/2.x/) |
| **Repository** | github.com/apache/logging-log4j2 (https://github.com/apache/logging-log4j2) |

## Apache Log4j 2

Apache Log4j 2 is the successor of Log4j 1 which was released as GA version in July 2015. The framework was rewritten from scratch and has been inspired by existing logging solutions, including Log4j 1 and java.util.logging. The main differences[14][15] from Log4j 1 are:

- Improved reliability. Messages are not lost while reconfiguring the framework like in Log4j 1 or Logback
- Extensibility: Log4j 2 supports a plugin system to let users define and configure custom components
- Simplified configuration syntax
- Support for xml, json, yaml and properties configurations
- Improved filters
- Property lookup support for values defined in the configuration file, system properties, environment variables, the ThreadContext Map, and data present in the event
- Support for multiple APIs: Log4j 2 can be used with applications using the Log4j 2, Log4j 1.2, SLF4J, Commons Logging and java.util.logging (JUL) APIs.
- Custom log levels
- Java 8-style lambda support for "lazy logging"
- Markers
- Support for user-defined Message objects
- "Garbage-free or low garbage" in common configurations
- Improved speed
- Improved support for Linux

One of the most recognized features of Log4j 2 is the performance of the "Asynchronous Loggers".[16] Log4j 2 makes use of the LMAX Disruptor.[17] The library reduces the need for kernel locking and increases the logging performance by a factor of 12. For example, in the same environment Log4j 2 can write more than 18,000,000 messages per second, whereas other frameworks like Logback and Log4j 1 just write < 2,000,000 messages per second.

## Features

### Log4j log levels

The following table defines the built-in log levels and messages in Log4j, in decreasing order of severity. The left column lists the log level designation in Log4j and the right column provides a brief description of each log level.

| Level | Description |
|---|---|
| **OFF** | The highest possible rank and is intended to turn off logging. |
| **FATAL** | Severe errors that cause premature termination. Expect these to be immediately visible on a status console. |
| **ERROR** | Other runtime errors or unexpected conditions. Expect these to be immediately visible on a status console. |
| **WARN** | Use of deprecated APIs, poor use of API, 'almost' errors, other runtime situations that are undesirable or unexpected, but not necessarily "wrong". Expect these to be immediately visible on a status console. |
| **INFO** | Interesting runtime events (startup/shutdown). Expect these to be immediately visible on a console, so be conservative and keep to a minimum. |
| **DEBUG** | Detailed information on the flow through the system. Expect these to be written to logs only. Generally speaking, most lines logged by your application should be written as DEBUG. |
| **TRACE** | Most detailed information. Expect these to be written to logs only. Since version 1.2.12.[18] |

### Custom log levels

Log4j 2 allows users to define their own log levels.[19] A source code generator tool is provided to create Loggers that support custom log levels identically to the built-in log levels. Custom log levels can either complement or replace the built-in log levels.

## Log4j configuration

Log4j can be configured[20] through a configuration file or through Java code. Configuration files can be written in XML, JSON, YAML, or properties file format. Three main components can be defined: Loggers, Appenders, and Layouts. Configuring logging via a file has the advantage that logging can be turned on or off without modifying the application that uses Log4j. The application can be allowed to run with logging off until there's a problem, for example, and then logging can be turned back on simply by modifying the configuration file.

**Loggers**[21] are named log message destinations. They are the names that are known to the Java application. Each logger is independently configurable as to what level of logging (FATAL, ERROR, etc.) it currently logs. In early versions of Log4j, these were called category and priority, but now they're called logger and level, respectively. A Logger can send log messages to multiple Appenders.

The actual outputs are done by **Appenders**.[22] There are numerous Appenders available, with descriptive names, such as FileAppender, RollingFileAppender, ConsoleAppender, SocketAppender, SyslogAppender, and SMTPAppender. Log4j 2 added Appenders that write to Apache Flume, the Java Persistence API, Apache Kafka, NoSQL databases, Memory-mapped files, Random Access files[23] and ZeroMQ endpoints. Multiple Appenders can be attached to any Logger, so it's possible to log the same information to multiple outputs; for example to a file locally and to a socket listener on another computer.

Appenders use **Layouts**[24] to format log entries. A popular way to format one-line-at-a-time log files is PatternLayout, which uses a pattern string, much like the C / C++ function printf. There are also HTMLLayout and XMLLayout formatters for use when HTML or XML formats are more convenient, respectively. Log4j 2 added Layouts for CSV, Graylog Extended Log Format (GELF),[25] JSON, YAML and RFC-5424.[26]

In Log4j 2, **Filters**[27] can be defined on configuration elements to give more fine-grained control over which log entries should be processed by which Loggers and Appenders. In addition to filtering by log level and regular expression matching on the message string, Log4j 2 added burst filters, time filters, filtering by other log event attributes like Markers or Thread Context Map and JSR 223 script filters.

To debug a misbehaving configuration:

- In Log4j 2 configurations set the `status` attribute to TRACE to send internal status logging output to standard out. To enable status logging before the configuration is found, use the Java VM property `-Dorg.apache.logging.log4j.simplelog.StatusLogger.level=trace`.
- In Log4j 1, use the Java VM property `-Dlog4j.debug`.

To find out where a log4j2.xml configuration file was loaded from inspect `getClass().getResource("/log4j2.xml")`.

There is also an implicit "unconfigured" or "default" configuration of Log4j, that of a Log4j-instrumented Java application which lacks any Log4j configuration. This prints to stdout a warning that the program is unconfigured, and the URL to the Log4j web site where details on the warning and configuration may be found. As well as printing this warning, an unconfigured Log4j application will only print ERROR or FATAL log entries to standard out.

### Example for Log4j 2

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="trace" monitorInterval="60">
  <Properties>
    <Property name="filename">target/test.log</Property>
  </Properties>

  <Appenders>
    <Console name="STDOUT">
      <PatternLayout pattern="%d %p %c{1.} [%t] %m%n"/>
    </Console>

    <File name="file" fileName="${filename}">
      <PatternLayout>
        <pattern>%d %p %c{1.} [%t] %m%n</pattern>
      </PatternLayout>
    </File>
  </Appenders>

  <Loggers>
    <!--
         loggers whose name starts with 'org.springframework' will only log messages of level "info" or higher;
```

```
        if you retrieve Loggers by using the class name (e.g. Logger.getLogger(AClass.class))
        and if AClass is part of the org.springframework package, it will belong to this category
    -->
    <Logger name="org.springframework" level="info" additivity="false" />

    <!--
        Filter example: for loggers whose name starts with 'com.mycompany.myproduct',
        log entries of level "debug" or higher whose ThreadContextMap data contains
        the key-value pair "test=123", also send these log entries to the "STDOUT" appender.
    -->
    <Logger name="com.mycompany.myproduct" level="debug" additivity="true">
      <ThreadContextMapFilter>
        <KeyValuePair key="test" value="123"/>
      </ThreadContextMapFilter>
      <AppenderRef ref="STDOUT"/>
    </Logger>

    <!--
        By default, all log messages of level "trace" or higher will be logged.
        Log messages are sent to the "file" appender and
        log messages of level "error" and higher will be sent to the "STDOUT" appender.
    -->
    <Root level="trace">
      <AppenderRef ref="file"/>
      <AppenderRef ref="STDOUT" level="error"/>
    </Root>
  </Loggers>

</Configuration>
```

## Example for Log4j 1.2

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE log4j:configuration PUBLIC "-//LOGGER" "http://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/xml/doc-files/log4j.dtd">
<log4j:configuration>
    <!--
        an appender is an output destination, such as the console or a file;
        names of appenders are arbitrarily chosen.
    -->
    <appender name="stdout" class="org.apache.log4j.ConsoleAppender">
        <layout class="org.apache.log4j.PatternLayout">
            <param name="ConversionPattern"
                value="%d{ABSOLUTE} %5p %c{1}:%L - %m%n" />
        </layout>
    </appender>

    <!--
        loggers of category 'org.springframework' will only log messages of level "info" or higher;
        if you retrieve Loggers by using the class name (e.g. Logger.getLogger(AClass.class))
        and if AClass is part of the org.springframework package, it will belong to this category
    -->
    <logger name="org.springframework">
        <level value="info"/>
    </logger>

    <!--
        everything of spring was set to "info" but for class
        PropertyEditorRegistrySupport we want "debug" logging
    -->
    <logger name="org.springframework.beans.PropertyEditorRegistrySupport">
        <level value="debug"/>
    </logger>

    <logger name="org.acegisecurity">
        <level value="info"/>
    </logger>


    <root>
        <!--
            all log messages of level "debug" or higher will be logged, unless defined otherwise
            all log messages will be logged to the appender "stdout", unless defined otherwise
        -->
        <level value="debug" />
        <appender-ref ref="stdout" />
    </root>
</log4j:configuration>
```

## TTCC

TTCC is a message format used by log4j.[28] TTCC is an acronym for *Time Thread Category Component*. It uses the following pattern:

```
%r [%t] %-5p %c %x - %m%n
```

Where

| Mnemonic | Description |
|---|---|
| %r | Used to output the number of milliseconds elapsed from the construction of the layout until the creation of the logging event. |
| %t | Used to output the name of the thread that generated the logging event. |
| %p | Used to output the priority of the logging event. |
| %c | Used to output the category of the logging event. |
| %x | Used to output the NDC (nested diagnostic context) associated with the thread that generated the logging event.[29] |
| %X{key} | Used to output the MDC (mapped diagnostic context) associated with the thread that generated the logging event for specified key.[30] |
| %m | Used to output the application supplied message associated with the logging event. |
| %n | Used to output the platform-specific newline character or characters. |

### Example output

```
467 [main] INFO  org.apache.log4j.examples.Sort - Exiting main method.
```

## Ports

- **log4c** – A port for C. *Log4C* is a C-based logging library, released on SourceForge under the LGPL license. For various Unix operating systems the autoconf and automake files are provided. On Windows a Makefile is provided for use with MSVC. Developers may also choose to use their own make system to compile the source, depending on their build engineering requirements. An instance of the *log4c* library may be configured via three methods: using environment variables, programmatically, or via XML configuration file. log4c has appenders for Files, Streams and memory mapped files. (No Socket Adapter.) Last version is 1.2.4, released in 2013, and the project is no longer actively developed.[31]
- **log4js** – A port for JavaScript. Log4js is available under the licence of Apache Software Foundation. One special feature of Log4js is the ability to log the events of the browser remotely on the server. Using Ajax it is possible to send the logging events in several formats (XML, JSON, plain ASCII, etc.) to the server to be evaluated there. The following appenders are implemented for *log4js*: AjaxAppender, ConsoleAppender, FileAppender, JSConsoleAppender, MetatagAppender, and WindowsEventsAppender. The following Layout classes are provided: BasicLayout, HtmlLayout, JSONLayout, and XMLLayout. Latest version as of 2021 was 2.0 of 18 October 2018.[32]
- **log4javascript** – Another port for JavaScript. log4javascript is a JavaScript logging framework based on the *log4j*. The latest version as of 2021 was 1.4.13, released on 23 May 2015.[33]
- **JSNLog** – A port for JavaScript. Automatically places messages from JavaScript loggers in server side logs using a .NET server side component that interfaces with Log4Net, NLog, Elmah or Common.Logging. This to provide an integrated log for client and server side events. Request ids correlate events related to a specific user. Configuration is via a server side web.config file. Supports exception logging including stack traces. As of 2021 the latest version was 3.0.0 and updates were made regularly.[34]
- **Apache Log4net** – A port to the Microsoft .NET Framework. The initial work was done by Neoworks and was donated to the Apache Software Foundation in February 2004. The framework is similar to the original log4j while taking advantage of new features in the .NET runtime. Provides Nested Diagnostic Context (NDC) and Mapped Diagnostic Context (MDC). The latest version as of 2022 was 2.0.15.[35]
- **log4perl** – A Perl port of log4j. Version 1.57 of October 2022 was current as of 2024.[36]
- **Apache log4php** – "A versatile logging framework for PHP. Originally a port of Apache log4j to PHP, it has grown to include various PHP specific features."[37]
- **PL-SQL-Logging-Utility** is an adaptation of log4j in PL/SQL.[38]
- **Log4db2** is a logging utility for DB2 for LUW that uses SQL instructions with SQL PL code.[39]
- **Apache Log4cxx** – A logging framework for C++ patterned after Apache log4j, which uses Apache Portable Runtime for most platform-specific code and should be usable on any platform supported by APR. The latest version is 0.13.0, released in 2022.[40]
- **Log4r** – A comprehensive and flexible logging library written in Ruby for use in Ruby programs. It was inspired by and provides much of the features of the Apache Log4j project.[41]

## Log4Shell vulnerability

A zero-day vulnerability involving remote code execution in Log4j 2, given the descriptor "Log4Shell" (CVE-2021-44228), was found and reported to Apache by Alibaba on November 24, 2021, and published in a tweet on December 9, 2021.[12] Affected services include Cloudflare, iCloud, *Minecraft: Java Edition*,[42] Steam, Tencent QQ, and Twitter.[43][44][45] The Apache Software Foundation assigned the maximum CVSS severity rating of 10 to Log4Shell, as millions of servers could be potentially vulnerable to the exploit.[45] The vulnerability was characterized by cybersecurity firm Tenable as "the single biggest, most critical vulnerability of the last decade"[13] and Lunasec's Free Wortley characterized it as "a design failure of catastrophic proportions".[46]

In the United States, the director of the Cybersecurity and Infrastructure Security Agency (CISA), Jen Easterly, termed the exploit "critical" and advised vendors to prioritize software updates,[47] and the German agency Federal Office for Information Security (BSI) designated the exploit as being at its highest threat level, calling it an "extremely critical threat situation" (translated).[48][49] The Canadian Centre for Cyber Security (CCCS) called on organisations to take on immediate action.[50]

The feature causing the vulnerability could be disabled with a configuration setting, which had been removed[51] in Log4j version 2.15.0-rc1 (officially released on December 6, 2021, three days before the vulnerability was published), and replaced by various settings restricting remote lookups, thereby mitigating the vulnerability.[52][53] For additional security, from version 2.16.0 onward, all features using JNDI, on which this vulnerability was based, were disabled by default, and support for message lookups removed.[54][55]

## See also

Computer programming portal
Free and open-source software portal

# References


1. "Apache Log4j 1.2 Release History" (https://logging.apache.org/log4j/1.2/changes-report.html). *apache.org*. Apache Software Foundation. Retrieved September 2, 2014.
2. "Release 2.26.0" (https://github.com/apache/logging-log4j2/releases/tag/rel/2.26.0). May 7, 2026. Retrieved May 7, 2026.
3. "Release Notes - Log4j" (https://logging.apache.org/log4j/2.x/release-notes/index.html). *logging.apache.org*. Retrieved October 9, 2023.
4. "Reload4j Home" (http://reload4j.qos.ch/). reload4j.qos.ch. Retrieved January 12, 2022.
5. "Logback Home" (http://logback.qos.ch/). Logback.qos.ch. Retrieved July 24, 2014.
6. Grigg, Kadi (February 4, 2022). "Wicked Good Development - Cybersecurity Experts Talk Log4J, Open Source and More" (https://blog.sonatype.com/wicked-good-development-episode-1). *blog.sonatype.com*. Retrieved August 16, 2022.
7. "Log4j 2 Guide - Apache Log4j 2" (https://logging.apache.org/log4j/2.x/). Logging.apache.org. July 12, 2014. Retrieved July 24, 2014.
8. Goers, Ralph (December 15, 2019). "Why was Log4j 2 created?" (https://www.ralphgoers.com/post/why-was-log4j-2-created). *Ralph Goers*.
9. "Log4j 2 Guide - Apache Log4j 2: News" (https://logging.apache.org/log4j/2.x/index.html#News). Logging.apache.org. July 12, 2014. Retrieved July 24, 2014.
10. "Apache Logging Services Project Announces Log4j 1 End-Of-Life; Recommends Upgrade to Log4j 2" (https://blogs.apache.org/foundation/entry/apache_logging_services_project_announces). blogs.apache.org. August 5, 2015. Retrieved July 3, 2016.
11. "Reload4j Project; Easy migration from log4j 1.2.x" (https://reload4j.qos.ch). qos.ch. January 12, 2022. Retrieved January 12, 2022.
12. "What's the Deal with the Log4Shell Security Nightmare?" (https://www.lawfaremedia.org/article/whats-deal-log4shell-security-nightmare). *Lawfare*. December 10, 2021.
13. "Recently uncovered software flaw 'most critical vulnerability of the last decade' " (https://www.theguardian.com/technology/2021/dec/10/software-flaw-most-critical-vulnerability-log-4-shell). *the Guardian*. Associated Press. December 11, 2021.
14. "The new log4j 2.0" (http://www.grobmeier.de/the-new-log4j-2-0-05122012.html). Grobmeier.de. December 5, 2012. Retrieved July 24, 2014.
15. "Log4j – Overview - Apache Log4j 2" (https://logging.apache.org/log4j/2.x/manual/index.html). logging.apache.org. June 5, 2016. Retrieved July 3, 2016.
16. "Log4j 2 Asynchronous Loggers for Low-Latency Logging - Apache Log4j 2" (https://logging.apache.org/log4j/2.x/manual/async.html#Performance). Logging.apache.org. July 12, 2014. Retrieved July 24, 2014.
17. "Disruptor by LMAX-Exchange" (https://lmax-exchange.github.io/disruptor/). Lmax-exchange.github.io. Retrieved July 24, 2014.
18. "Level (Apache Log4j 1.2.17 API)" (https://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/Level.html#TRACE). Logging.apache.org. June 9, 2012. Retrieved July 24, 2014.
19. "Custom Log Levels" (https://logging.apache.org/log4j/2.x/manual/customloglevels.html). Logging.apache.org. July 12, 2014. Retrieved July 16, 2016.
20. "Configuration" (https://logging.apache.org/log4j/2.x/manual/configuration.html). Logging.apache.org. July 5, 2016. Retrieved July 16, 2016.
21. "Architecture" (https://logging.apache.org/log4j/2.x/manual/architecture.html). Logging.apache.org. July 5, 2016. Retrieved July 16, 2016.
22. "Appenders" (https://logging.apache.org/log4j/2.x/manual/appenders.html). Logging.apache.org. July 5, 2016. Retrieved July 16, 2016.
23. "RandomAccessFile" (https://docs.oracle.com/javase/7/docs/api/java/io/RandomAccessFile.html). docs.oracle.com. July 28, 2011. Retrieved July 16, 2016.
24. "Layouts" (https://logging.apache.org/log4j/2.x/manual/layouts.html). Logging.apache.org. July 5, 2016. Retrieved July 16, 2016.
25. "GELF" (https://web.archive.org/web/20200209132812/http://docs.graylog.org/en/2.0/pages/gelf.html). docs.graylog.org. June 8, 2016. Archived from the original (http://docs.graylog.org/en/2.0/pages/gelf.html) on February 9, 2020. Retrieved July 16, 2016.
26. Gerhards, R. (March 1, 2009). "RFC 5424 - The Syslog Protocol" (https://tools.ietf.org/html/rfc5424). tools.ietf.org. doi:10.17487/RFC5424 (https://doi.org/10.17487%2FRFC5424). Retrieved July 16, 2016. `{{cite journal}}`: Cite journal requires `|journal=` (help)
27. "Filters" (https://logging.apache.org/log4j/2.x/manual/filters.html). Logging.apache.org. July 5, 2016. Retrieved July 16, 2016.
28. "TTCCLayout (Apache Log4j 1.2.17 API)" (https://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/TTCCLayout.html). Logging.apache.org. June 9, 2012. Retrieved July 24, 2014.
29. "Class NDC" (https://people.apache.org/~carnold/log4j/docs/org/apache/log4j/NDC.html). Archived (https://web.archive.org/web/20070820182511/http://logging.apache.org/log4j/docs/api/org/apache/log4j/NDC.html) from the original on August 20, 2007. Retrieved December 11, 2021.
30. "MDC (Apache Log4j 1.2.17 API)" (https://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/MDC.html). Logging.apache.org. June 9, 2012. Retrieved July 24, 2014.
31. "Logging Framework for C | Free System Administration software downloads at" (https://sourceforge.net/projects/log4c/). Sourceforge.net. Retrieved July 24, 2014.
32. "stritti/Log4js - The Logging Framework for JavaScript with no runtime dependencies" (https://github.com/stritti/log4js). *GitHub*. Retrieved December 11, 2021.
33. "a JavaScript logging framework" (http://log4javascript.org/). log4javascript. Retrieved December 11, 2021.

34. "Logging JavaScript errors to your server side log" (http://jsnlog.com/). JSNLog. Retrieved December 11, 2021.
35. "Apache log4net" (https://logging.apache.org/log4net/). Logging.apache.org. Retrieved December 11, 2021.
36. Schilli, Mike; Goess, Kevin. "log4perl - log4j for Perl" (http://mschilli.github.io/log4perl/). *log4perl*. Retrieved December 11, 2021.
37. "Apache Logging Services" (https://logging.apache.org/). Apache.org. Retrieved March 11, 2015.
38. "tmuth/Logger-A-PL-SQL-Logging-Utility — GitHub" (https://github.com/tmuth/Logger---A-PL-SQL-Logging-Utility). Github.com. Retrieved July 24, 2014.
39. "Log4db2 by angoca" (https://angoca.github.io/log4db2/). Angoca.github.io. Retrieved July 24, 2014.
40. "log4cxx - Changelog" (https://logging.apache.org/log4cxx/latest_stable/changelog.html). *logging.apache.org*.
41. "Log4r Manual" (http://log4r.rubyforge.org/). log4r.rubyforge.org. Retrieved April 13, 2017.
42. "Security Vulnerability in Minecraft: Java Edition" (https://help.minecraft.net/hc/en-us/articles/4416199399693-Security-Vulnerability-in-Minecraft-Java-Edition). *Minecraft*. December 10, 2021.
43. Goodin, Dan (December 9, 2021). "Zeroday in ubiquitous Log4j tool poses a grave threat to the Internet" (https://arstechnica.com/information-technology/2021/12/minecraft-and-other-apps-face-serious-threat-from-new-code-execution-bug/). *Ars Technica*. Retrieved December 10, 2021.
44. "Worst Apache Log4j RCE Zero day Dropped on Internet" (https://www.cyberkendra.com/2021/12/worst-log4j-rce-zeroday-dropped-on.html). *Cyber Kendra*. December 9, 2021. Retrieved December 10, 2021.
45. Mott, Nathaniel (December 10, 2021). "Countless Servers Are Vulnerable to Apache Log4j Zero-Day Exploit" (https://www.pcmag.com/news/countless-serves-are-vulnerable-to-apache-log4j-zero-day-exploit). *PC Magazine*. Retrieved December 10, 2021.
46. Newman, Lily Hay (December 10, 2021). "The Internet Is on Fire" (https://www.wired.com/story/log4j-flaw-hacking-internet/). *Wired* – via www.wired.com.
47. "Statement from CISA Director Easterly on "Log4j" Vulnerability" (https://www.cisa.gov/news/2021/12/11/statement-cisa-director-easterly-log4j-vulnerability). *CISA*. Washington. December 11, 2021.
48. "BSI warnt vor Sicherheitslücke" (https://www.tagesschau.de/inland/bsi-schadsoftware-101.html) [BSI warns of security vulnerabilities]. *Tagesschau* (in German). December 12, 2021.
49. "Warnstufe Rot: Schwachstelle Log4Shell führt zu extrem kritischer Bedrohungslage" (https://www.bsi.bund.de/DE/Service-Navi/Presse/Pressemitteilungen/Presse2021/211211_log4Shell_WarnstufeRot.html) [Red alarm: Log4Shell vulnerability causes extremely critical threat situation]. *BSI press service* (in German). December 12, 2021.
50. "Statement from the Minister of National Defence on Apache Vulnerability and Call to Canadian Organizations to Take Urgent Action" (https://web.archive.org/web/20211220032436/https://www.cyber.gc.ca/en/news/statement-minister-national-defence-apache-vulnerability). *Government of Canada*. December 12, 2021. Archived from the original (https://cyber.gc.ca/en/news/statement-minister-national-defence-apache-vulnerability) on December 20, 2021. Retrieved December 12, 2021.
51. "LOG4J2-3198: Log4j2 no longer formats lookups in messages by default" (https://github.com/apache/logging-log4j2/commit/001aaada7dab82c3c09cde5f8e14245dc9d8b454). *GitHub*. December 5, 2021.
52. "Restrict LDAP access via JNDI by rgoers · Pull Request #608 · apache/logging-log4j2" (https://github.com/apache/logging-log4j2/pull/608). *GitHub*. 30 November–5 December 2021
53. "Apache Log4j Security Vulnerabilities" (https://logging.apache.org/log4j/2.x/security.html). December 6, 2021.
54. "LOG4J2-3208: Disable JNDI by default" (https://issues.apache.org/jira/browse/LOG4J2-3208). December 11, 2021. Retrieved December 14, 2021.
55. "LOG4J2-3211: Remove support for Lookups in messages" (https://issues.apache.org/jira/browse/LOG4J2-3211). December 13, 2021. Retrieved December 14, 2021.


## Further reading


- Gülcü, Ceki (February 2010), *The Complete Log4j Manual* (2nd ed.), QOS.ch, p. 204, ISBN 978-2-9700369-0-6
- Gupta, Samudra (June 22, 2005), *Pro Apache Log4j* (2nd ed.), Apress, p. 224, ISBN 978-1-59059-499-5


## External links

- Official website (https://logging.apache.org/log4j/2.x/)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Log4j&oldid=1355928727"